END-TO-END MLOps FOR HEART DISEASE PREDICTION

Machine Learning Operations — Assignment 01

Submission field	Details
Student name	Avadhanam Koushik Sai Santhosh
Roll number / ID	2025AE05374
Course	Machine Learning Operations (MLOps) AIMLCZG523
Repository	https://github.com/KoushikAvadhanam/heart-disease-mlops
Deployment environment	Local Kubernetes using Docker Desktop
API access	Local port-forward instructions; not a public endpoint
Status	Final report — reviewed for submission

Abstract

This project implements an end-to-end machine-learning workflow for binary heart-disease classification using the UCI Heart Disease dataset. The workflow includes data acquisition, exploratory data analysis, preprocessing, model comparison, MLflow experiment tracking, model serialization, a FastAPI prediction service, Docker packaging, automated testing through GitHub Actions, Kubernetes deployment, and Prometheus/Grafana monitoring.

The dataset contains 303 records and 14 original columns. The target is converted to a binary label: 0 represents no disease and values greater than 0 represent disease present. Logistic Regression and Random Forest are evaluated using stratified five-fold cross-validation and a held-out test set. The API is deployed locally in Kubernetes with two replicas, while Prometheus scrapes the API metrics endpoint and Grafana visualizes availability and process telemetry.

Objectives

- Build a reproducible data and model-training pipeline.
- Compare two classification algorithms with appropriate metrics.
- Track experiments and artifacts using MLflow.
- Serve predictions through a containerized API.
- Automate checks and demonstrate local Kubernetes deployment and monitoring.

1. Project Overview and Architecture

The assignment demonstrates MLOps practices across the model lifecycle, rather than only training a classifier in a notebook. The repository separates data, model artifacts, application code, tests, CI configuration, deployment manifests, reports, and screenshots.

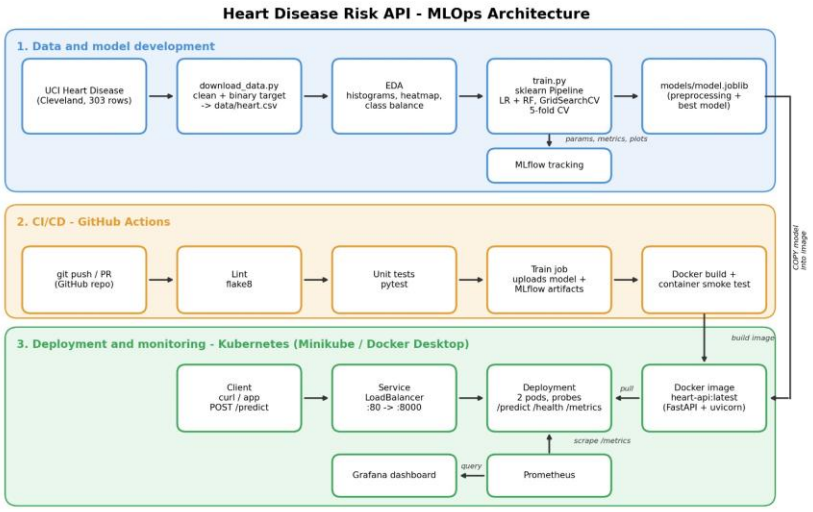
Architecture components

Component	Responsibility
UCI Heart Disease data	Source records used for exploratory analysis and supervised learning.
EDA/training scripts	Inspect data, preprocess features, train models, evaluate metrics, save plots/results.
MLflow	Record parameters, metrics, plots, reports, and fitted model artifacts.
Serialized pipeline	Reusable preprocessing-plus-model artifact at models/heart_disease_pipeline.joblib.
FastAPI application	Expose health, prediction, and metrics endpoints.
Docker	Package API and model into a repeatable runtime image.
GitHub Actions	Run automated checks, tests, training, and container build jobs.
Kubernetes	Run two API replicas and expose the application through a Service.
Prometheus/Grafana	Collect API metrics and display availability/process telemetry.

Architecture flow

Data → preprocessing and training → MLflow experiment records → serialized model pipeline → Docker image → Kubernetes Deployment and Service → /metrics endpoint → Prometheus → Grafana dashboard. GitHub Actions validates code and builds/tests the project alongside this runtime flow.

The architecture diagram below summarizes the data/model workflow, CI/CD checks, and local deployment and monitoring components.



Repository

GitHub: https://github.com/KoushikAvadhanam/heart-disease-mlops

The repository contains source scripts, tests, CI workflow configuration, Docker files, Kubernetes manifests, EDA screenshots, model evaluation reports, and monitoring screenshots. Include the final report and demonstration video in the submission package.

2. Data Acquisition and EDA

The project uses the Heart Disease UCI dataset obtained from the UCI Machine Learning Repository. The downloaded CSV contains 303 rows and 14 columns. The original target column is named num. For this binary-classification task, target is defined as (num > 0).astype(int), and num is removed from the feature matrix to avoid target leakage.

Feature groups

Feature group	Columns used
Numeric	age, trestbps, chol, thalach, oldpeak
Categorical/coded	sex, cp, fbs, restecg, exang, slope, ca, thal
Target	target: 0 = no disease; 1 = disease present

Exploratory plots

- Class balance plot: inspect relative counts of target classes.
- Feature histograms: inspect distributions and possible skew/outliers.
- Correlation heatmap: review pairwise numeric associations; correlation is not causation.
- Missing-values plot: identify columns requiring imputation.

Preprocessing and interpretation

The pipeline handles missing numeric values using median imputation and missing categorical values using most-frequent imputation. Refer to the actual EDA output for missing-value counts rather than assuming values. Screenshots include class_balance.png, feature_histograms.png, correlation_heatmap.png, missing_values.png, and age_by_target.png.

The EDA plots support inspection of class balance, feature distributions, correlations, and missingness. Interpretations should remain descriptive: correlations do not establish causation, and this educational dataset is not sufficient for clinical conclusions.

3. Feature Engineering and Model Development

The training code uses a ColumnTransformer with separate numeric and categorical pipelines. Numeric fields are median-imputed and standardized using StandardScaler. Categorical fields are imputed with the most frequent value and one-hot encoded with handle_unknown='ignore'. Preprocessing is fitted inside the training pipeline to keep transformations consistent.

Train/test strategy

The data is split into 80% training and 20% held-out test data using random_state=42 and stratification. On the training partition, StratifiedKFold uses five folds with shuffling and random_state=42. The model-selection criterion is the highest mean cross-validation ROC-AUC.

Models compared

Model	Configuration in training code
Logistic Regression	max_iter=2000; random_state=42
Random Forest	n_estimators=300; min_samples_leaf=2; class_weight='balanced'; random_state=42

Recorded held-out metrics

Metric	Logistic Regression	Random Forest
Test Accuracy	88.52%	88.52%
Test Precision	83.87%	81.82%
Test Recall	92.86%	96.43%
Test ROC-AUC	96.65%	95.13%

Five-fold cross- validation results

Metric	Logistic Regression	Random Forest
CV Accuracy	84.71%	80.16%
CV Precision	87.68%	78.31%
CV Recall	78.34%	79.17%
CV ROC-AUC	90.25%	89.75%

These values reflect the recorded training run. Cross-validation means are distinct from held-out test metrics; the held-out test set is separate from five-fold cross-validation on the training partition.

Metric interpretation

Precision measures the fraction of positive predictions that are correct; recall measures the fraction of actual positive cases detected. In health-related screening, missed positive cases may matter, but false positives also have consequences. Threshold selection requires domain validation. This educational project is not a medical diagnostic tool.

4. Experiment Tracking with MLflow

MLflow is integrated into the training script with a local SQLite tracking store. The experiment is named 'Heart Disease Prediction'. The code starts a named run for each algorithm and logs parameters, cross-validation and held-out metrics, classification reports, confusion-matrix images, and the fitted pipeline.

Tracked information

- Model name, five-fold cross-validation, 20% test size, and random seed.
- Cross-validation accuracy, precision, recall, and ROC-AUC.
- Held-out test accuracy, precision, recall, and ROC-AUC.
- Classification report text and confusion-matrix plots.
- Fitted preprocessing/model pipeline artifact.

How to inspect experiments

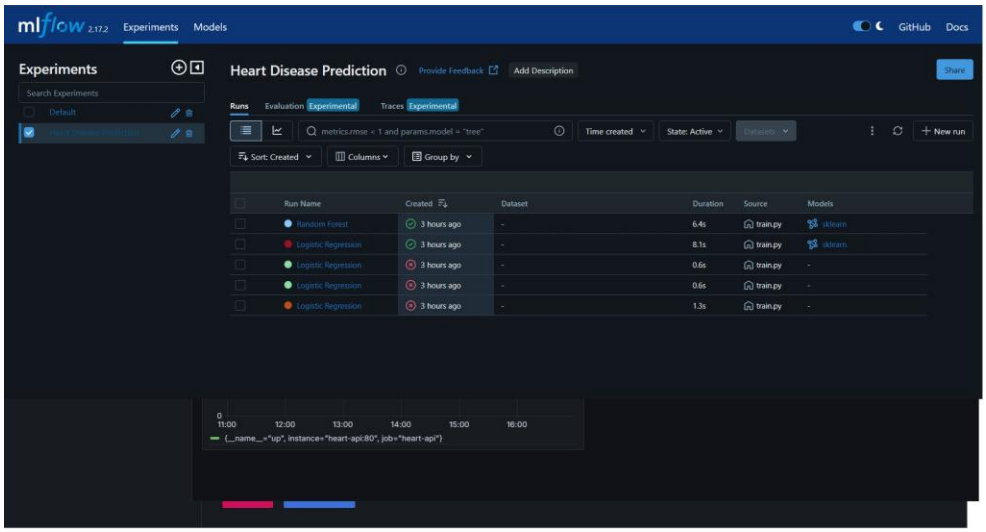
1. Activate the project's virtual environment.
2. Launch the MLflow UI using the tracking database and experiment generated by the training script.
3. Open the 'Heart Disease Prediction' experiment.
4. Compare Logistic Regression and Random Forest runs and inspect metrics/artifacts.
5. Capture a genuine MLflow UI screenshot showing the runs and metric comparison.

The screenshot below records the local MLflow interface with the Heart Disease Prediction experiment and its model runs. The tracking UI is local; it does not constitute a public endpoint.

Reproducibility

A fixed random seed and stratified folds make the experiment easier to reproduce. Results can still vary with dependency versions, so versions should remain aligned with the serialized model.

Figure: MLflow experiment tracking showing the Heart Disease Prediction experiment and recorded model runs.



5. Model Packaging, API, and Docker

The selected preprocessing-and-estimator pipeline is serialized to models/heart_disease_pipeline.joblib. Keeping preprocessing inside the saved pipeline means incoming API records receive the same transformations used during training.

API endpoints

Endpoint	Purpose	Evidence
GET /	Root/service information	Swagger UI at /docs
GET /health	Health and model-loaded status	Browser test and screenshot
POST /predict	Accept features and return prediction/confidence	Swagger test and screenshot
GET /metrics	Expose Prometheus-format telemetry	HTTP 200 and metric text confirmed

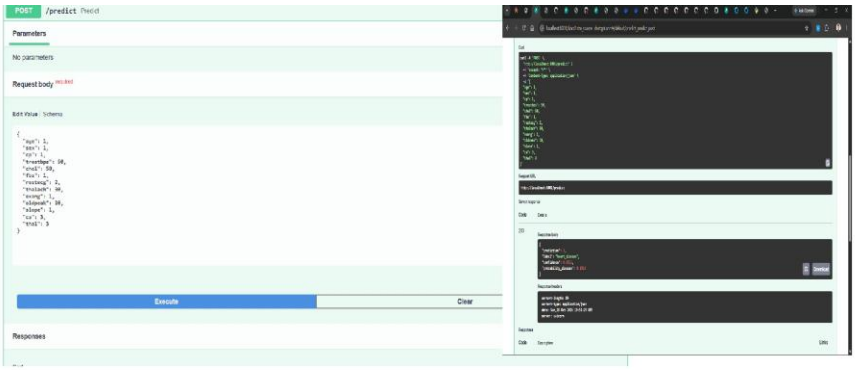
Docker packaging

The Docker image packages the API runtime and serialized model. The image uses Python 3.11 slim and sets MODEL_PATH to /app/models/heart_disease_pipeline.joblib. A prior container build and prediction test returned HTTP 200. The serving requirements pin scikit-learn to 1.9.1 to align with the saved model artifact.

Local testing

From the project root, build the image using the Dockerfile and run the container with the port mapping documented in README.md. Verify GET /health and use Swagger at /docs to submit a sample POST /predict request. If host port 8000 is in use, use a different host port or stop the previous container.

Document an actual prediction response from the API. Do not interpret its output as medical advice.





6. Automated Testing and CI/CD

The project includes automated tests and a GitHub Actions workflow. The intended stages include code quality checks, tests, model training, and Docker image build, helping detect regressions before changes are accepted.

Test coverage

- API root and health endpoint checks.
- Valid prediction and invalid-input validation.
- Model pipeline probability and prediction checks.
- Saved pipeline prediction consistency.
- Preprocessing checks for feature/target loading, imputation, scaling, and unseen categories.

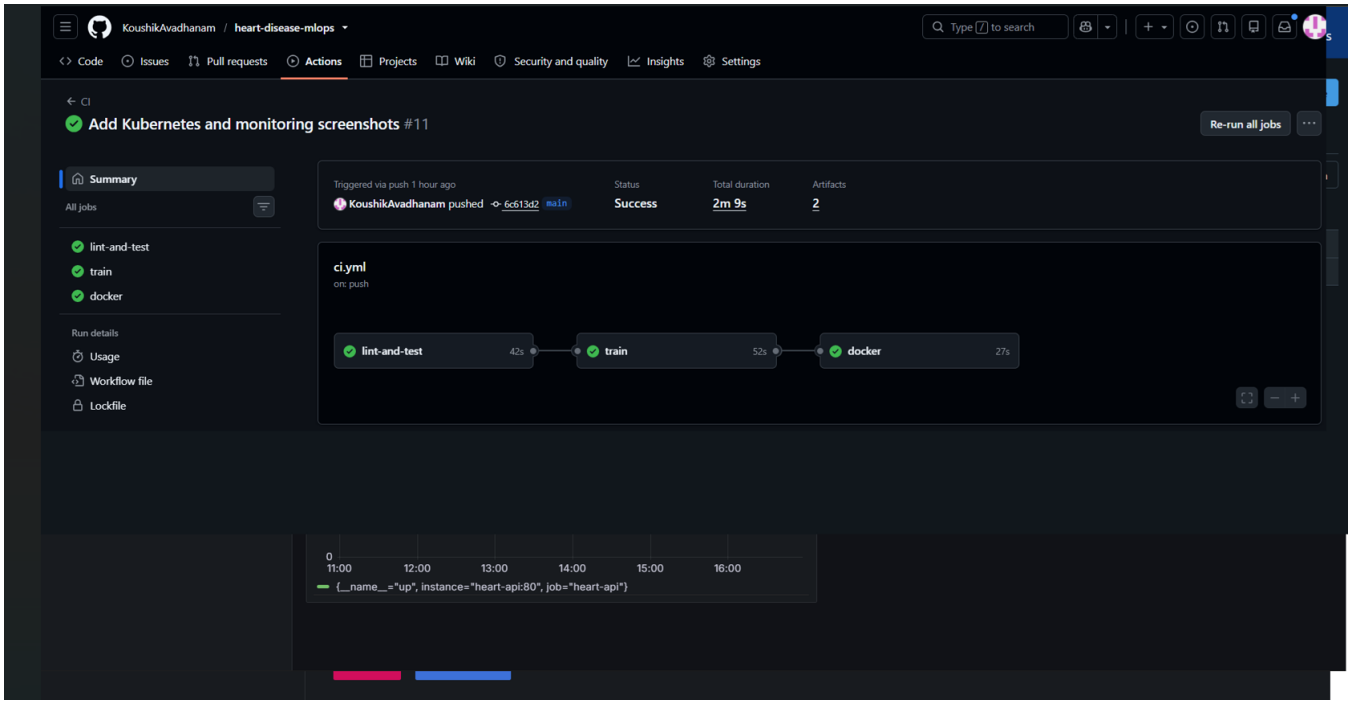
CI workflow

The latest observed GitHub Actions status showed lint-and-test, train, and Docker jobs green after serving dependency and model-path fixes. Include a screenshot from Actions showing the successful run and job names. The template also asks for a deliberately failing run and artifacts; include these only if genuine examples are available.

Evidence checklist

- Green GitHub Actions workflow with completed jobs.
- Pytest output showing passing tests.
- Docker build success and a successful API request.
- A genuine failed-run screenshot only if available; do not fabricate one.
- Uploaded workflow artifacts/logs if the workflow creates them.

A CI pipeline should fail when required checks fail and provide useful logs. Keep dependency versions consistent between local, Docker, and CI environments.



7. Kubernetes Deployment

The API is deployed to a local Kubernetes cluster created through Docker Desktop. The deployment manifest defines two API replicas, container port 8000, readiness/liveness checks against /health, and resource settings. A Kubernetes Service of type LoadBalancer exposes service port 80 and forwards to container target port 8000.

Verified cluster status

Resource	Observed state
heart-api Deployment	2/2 replicas available
heart-api Pods	Two pods Running, each 1/1 Ready
prometheus Deployment	1/1 available
grafana Deployment	1/1 available
heart-api Service	LoadBalancer; tested using port-forward

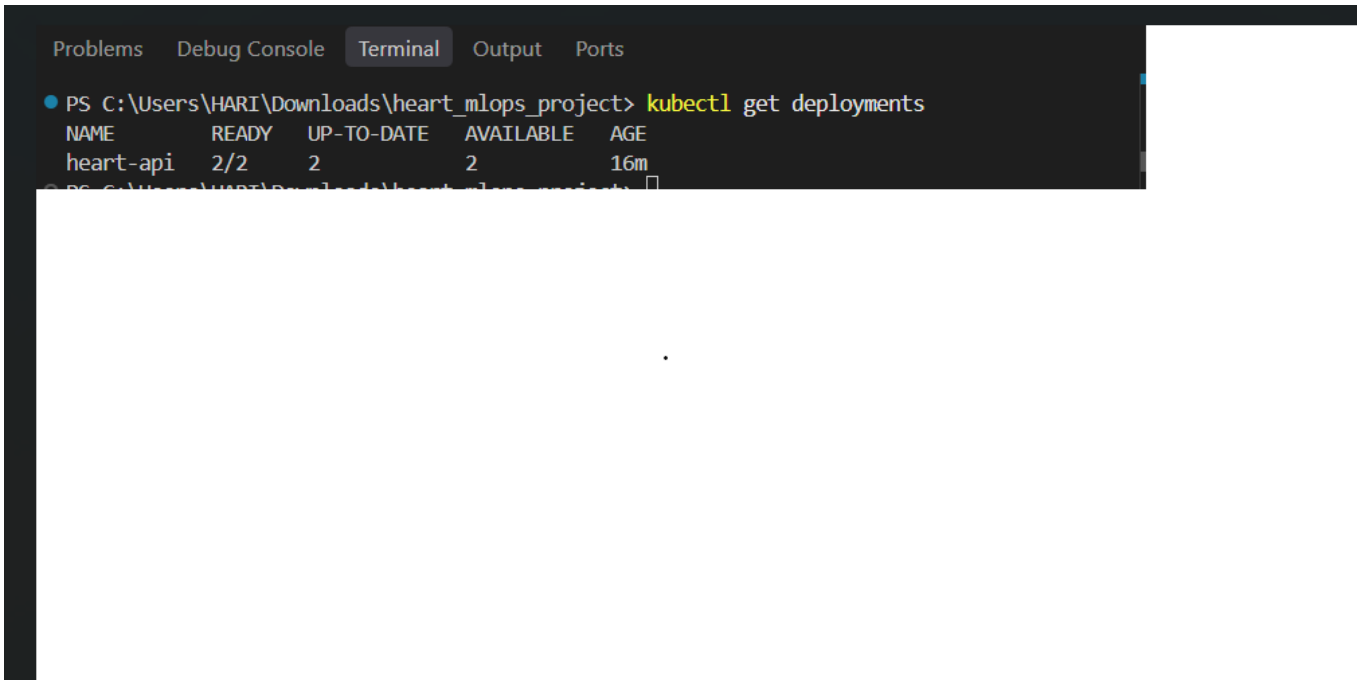
Local access

The verified local command was `kubectl port-forward service/heart-api 8001:80`. With the command running, health checks were available at http://localhost:8001/health and interactive API documentation at http://localhost:8001/docs. This is a local development endpoint, not a public cloud URL.

Useful verification commands

```
kubectl get pods
kubectl get deployments
kubectl get services
kubectl port-forward service/heart-api 8001:80
```

Evidence committed under screenshots/ includes kubernetes_deployment.png, api_health_check.png, and prediction_api_test.png. Local port-forward instructions require the cluster to be running.



8. Monitoring and Logging

The API's /metrics endpoint returned HTTP 200 and Prometheus text-format metrics, including Python runtime metrics. Prometheus was deployed with a ConfigMap scrape configuration targeting the heart-api Service at port 80. The Prometheus Targets page showed the API target as UP, and the `up` query returned value 1.

Grafana dashboard

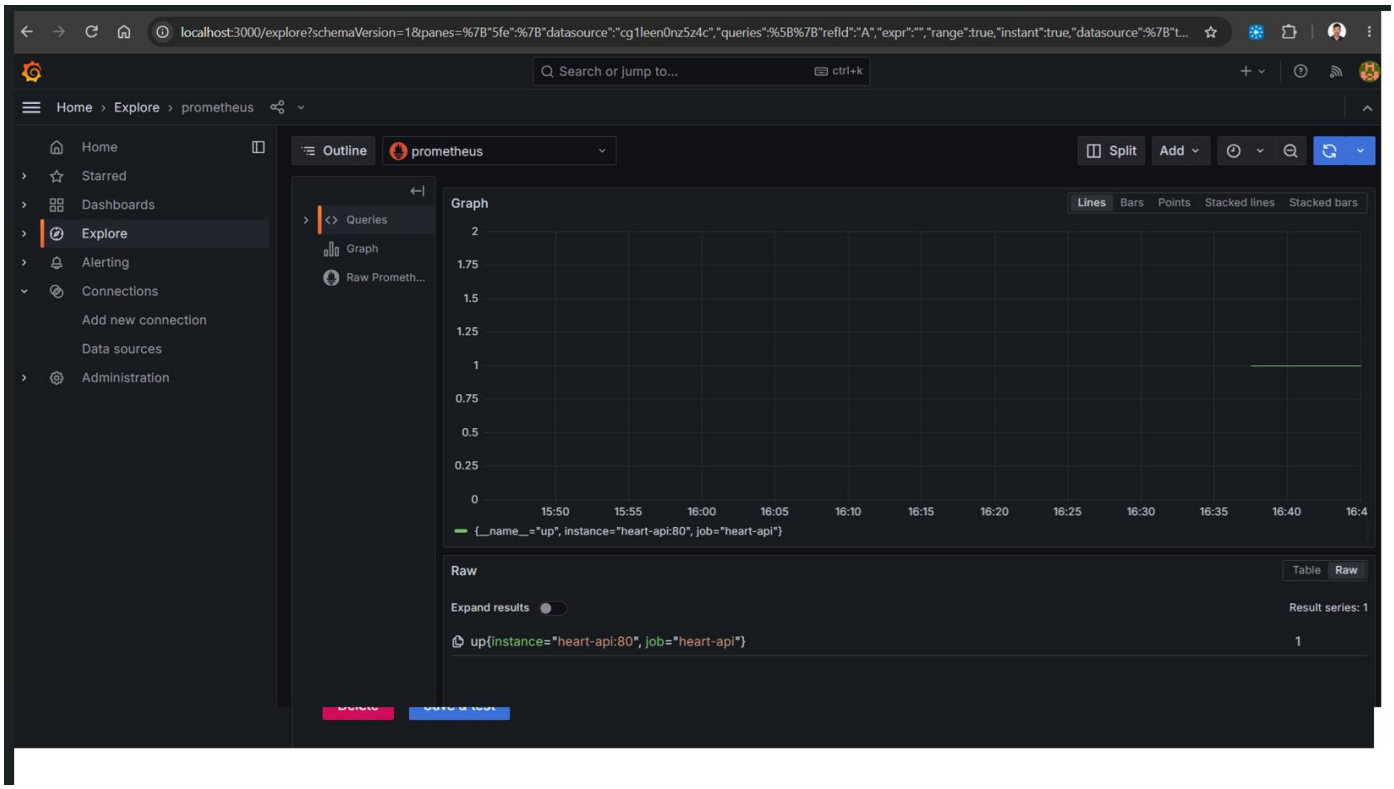
Grafana was deployed in the same Kubernetes cluster and configured with Prometheus as a data source. The 'Heart Disease API Monitoring' dashboard contains an availability panel using `up` and a process telemetry panel using `process_cpu_seconds_total`. The latter is accumulated CPU time, not an instantaneous CPU percentage.

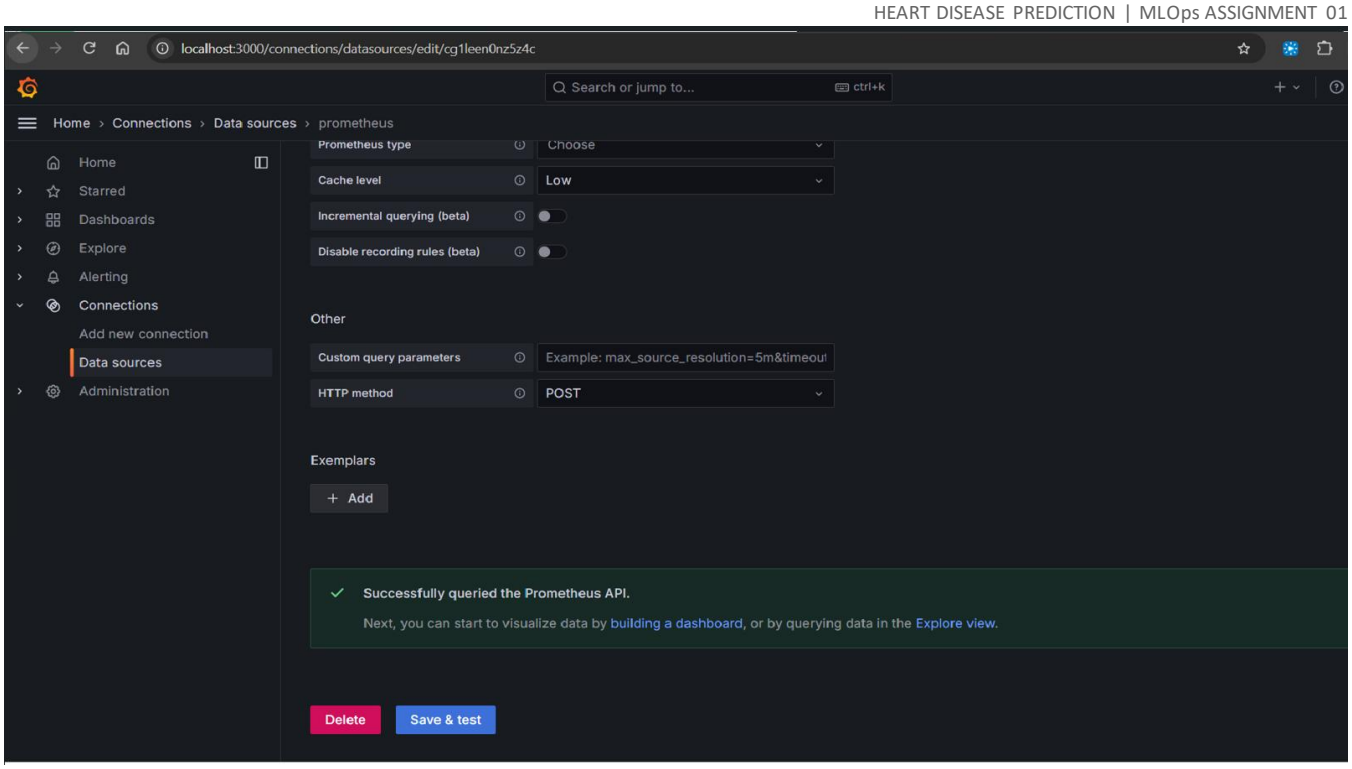
Monitoring evidence

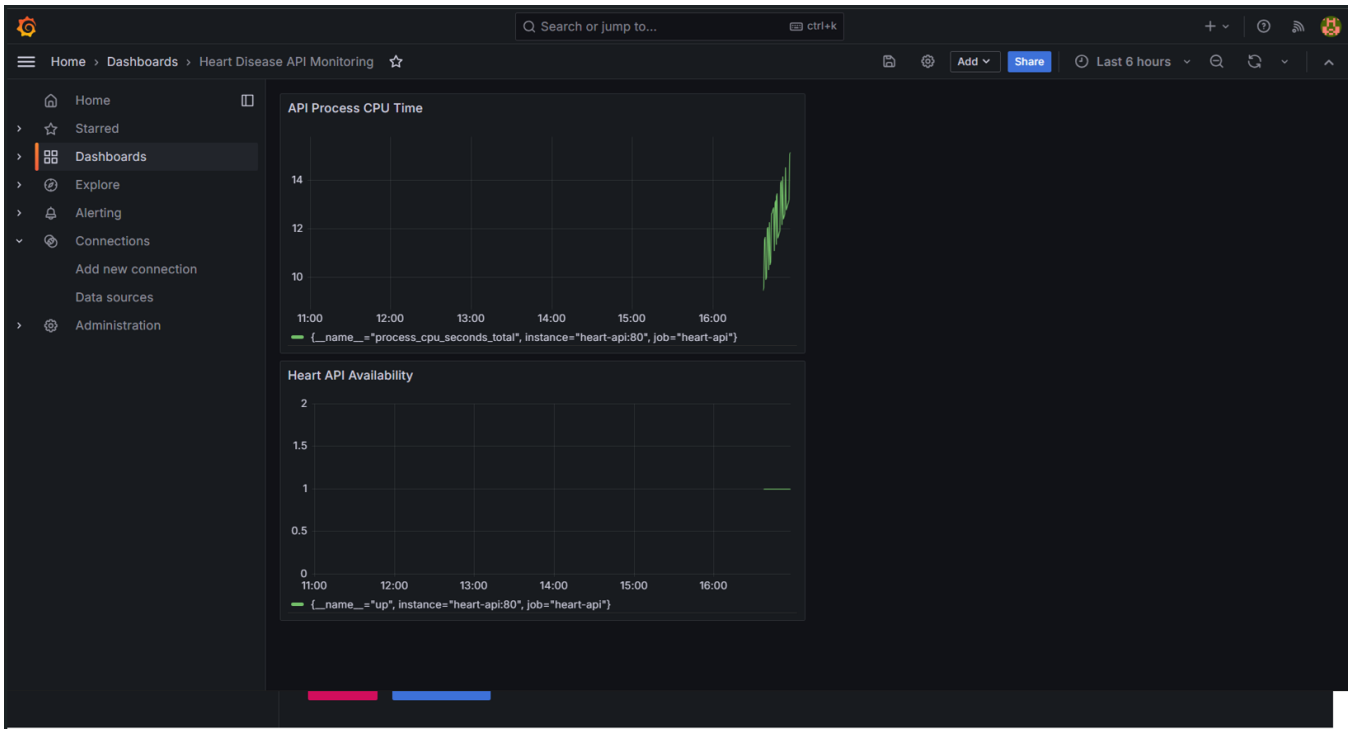
- screenshots/prometheus_target_up.png — target health query.
- screenshots/grafana_prometheus_connection.png — successful data-source test.
- screenshots/grafana_dashboard.png — dashboard panels.
- screenshots/api_health_check.png — API health endpoint.

Operational limitations

This is a local demonstration, not a hardened production observability stack. The dashboard demonstrates basic availability and process metrics. Production deployment would typically add authentication, TLS, resource dashboards, alert rules, persistent storage, retention policies, and log aggregation.




localhost:3000/connections/datasources/edit/cg1leen0nz5z4c
Search or jump to... ctrl+k
Home › Connections › Data sources › prometheus
Home
Starred
Dashboards
Explore
Alerting
Connections
Add new connection
Data sources
Administration
Prometheus type
Choose
Cache level
Low
Incremental querying (beta)
Disable recording rules (beta)
Other
Custom query parameters
Example: max_source_resolution=5m&timeout
HTTP method
POST
Exemplars
+ Add
Successfully queried the Prometheus API.
Next, you can start to visualize data by building a dashboard, or by querying data in the Explore view.
Delete
Save & test


Search or jump to... ctrl+k
Home › Dashboards › Heart Disease API Monitoring
Add
Share
Last 6 hours
Home
Starred
Dashboards
Explore
Alerting
Connections
Add new connection
Data sources
Administration
API Process CPU Time
14
12
10
11:00
12:00
13:00
14:00
15:00
16:00
{__name__="process_cpu_seconds_total", instance="heart-api:80", job="heart-api"}
Heart API Availability
2
1.5
1
0.5
0
11:00
12:00
13:00
14:00
15:00
16:00
{__name__="up", instance="heart-api:80", job="heart-api"}

9. Results, Limitations, and Conclusion

The project demonstrates an end-to-end workflow from dataset acquisition to a containerized prediction API running on Kubernetes. Two models were compared using stratified cross-validation and a held-out test set.

Results summary

The recorded Logistic Regression test results were accuracy 88.52%, precision 83.87%, recall 92.86%, and ROC-AUC 96.65%. Random Forest test results were accuracy 88.52%, precision 81.82%, recall 96.43%, and ROC-AUC 95.13%. Logistic Regression was selected by the mean cross-validation ROC-AUC criterion (90.25% versus 89.75% for Random Forest).

Automated checks were observed passing in the latest GitHub Actions run after correcting the serving dependency version and model path. The API health, prediction, and metrics endpoints were tested. Prometheus reported the API target as up, and Grafana successfully queried Prometheus.

Limitations and future work

- The dataset is small and should not be assumed representative of current clinical populations.
- A single held-out split can produce optimistic estimates and needs external validation.
- This is an educational demonstration and must not be used to diagnose or treat patients.
- Future work: calibration, threshold selection with clinical input, fairness checks, drift detection, alerting, and secured public deployment.
- Monitoring could add request rate, error rate, latency, memory, and container resource metrics.

Conclusion

This assignment demonstrates the main MLOps lifecycle components: reproducible preprocessing and training, experiment tracking, model packaging, automated checks, containerization, Kubernetes deployment, and basic monitoring. Validate every report claim against the repository and attach authentic evidence.

Setup and access instructions

6. Clone the repository and follow README.md to install dependencies.
7. Run the data download, EDA, and training scripts in the documented order.
8. Use Docker to build and test the serving image.
9. Enable Docker Desktop Kubernetes and apply manifests in k8s/.
10. Use kubectl port-forward to access the API, Prometheus, and Grafana locally.
11. Use screenshots as evidence and record a short end-to-end walkthrough.

Submission readiness

- Confirm the student name and roll number are correct.
- The model metrics in this report match the recorded model comparison results.
- Architecture and MLflow screenshots are included in this report.
- Commit and push the final report and evidence to the repository.
- Record and submit the short pipeline demonstration video.

Appendix — Evidence Index

Evidence	Repository path / note
Architecture	docs/architecture.png and docs/architecture.svg
EDA plots	screenshots/age_by_target.png, class_balance.png, correlation_heatmap.png, feature_histograms.png, missing_values.png
Model evaluation	reports/model_comparison.csv and confusion-matrix images
Kubernetes	screenshots/kubernetes_deployment.png
API health/prediction	screenshots/api_health_check.png and screenshots/prediction_api_test.png
Monitoring	screenshots/prometheus_target_up.png, screenshots/grafana_prometheus_connection.png, screenshots/grafana_dashboard.png
Repository	https://github.com/KoushikAvadhanam/heart-disease-mlops

Responsible-use statement

This classifier is an educational machine-learning demonstration built from a public dataset. It has not been clinically validated or approved for real-world medical use. Predictions must not be used for diagnosis, treatment, or health decisions.

Student review note

Review this draft, replace placeholders, verify numerical results and implementation details, insert authentic figures/screenshots, and be prepared to explain the design decisions independently.